Documentation technique — Mini Dashboard

Tableau de bord LVGL sur Raspberry Pi 5 — Version 4

`lv_port_vscode/src` | `NavRelay` | `SETUP.sh`

meka3

8 novembre 2026

Table des matières

1 DÉMARRAGE RAPIDE (NOUVELLE INSTALLATION) 4
1.1 Installation en 3 etapes . . . 4

2 Nouveautés de la version 4 5

3 Vue d'ensemble du système 6
3.1 Architecture globale . . . 6
3.2 Prérequis matériels et logiciels . . . 6
3.2.1 Build tools (obligatoire pour compiler) . . . 6
3.2.2 Paquets intime et dépendances . . . 6
3.2.3 Installation automatisée (RECOMMANDÉE) . . . 7

4 Compilation et mise en place du projet LVGL 7
4.1 Cloner le depôt et initialiser les sous-modules . . . 7
4.2 Structure du repertoire `lv_port_vscode` . . . 8
4.3 Configuration LVGL (`lv_conf.h`) . . . 8
4.4 Compilation complete avec CMake (RECOMMANDÉE) . . . 8
4.5 Compilation manuelle (alternative) . . . 9
4.6 Lancer le dashboard . . . 9
4.7 Arrêt propre du dashboard . . . 9

5 Pair Button et agent d'appairage Bluetooth 9
5.1 Principe : un agent BlueZ integre au dashboard . . . 9
5.2 Fenetre d'appairage (bouton Pair) . . . 10
5.3 Regles d'acceptation . . . 10
5.4 Parametres (`bt_agent.c`) . . . 10
5.5 Popup « Pairing successful » du bureau Pi OS . . . 11
5.6 Appairage initial du telephone . . . 11
5.6.1 Methode 1 : Via le bouton Pair du dashboard (RECOMMANDÉE) . . . 11
5.6.2 Methode 2 : En ligne de commande (manuel) . . . 11

6 Configuration systeme Bluetooth 12
6.1 Configuration BlueZ pour la musique et les appels . . . 12
6.1.1 Fichier `main.conf` ecrit par `SETUP.sh` . . . 12
6.1.2 Groupes de l'utilisateur . . . 12
6.1.3 Activation du service oFono . . . 12
6.1.4 Redemarrage des services Bluetooth . . . 12
6.2 Configuration PipeWire pour HFP via oFono . . . 13

6.3 Verification de la pile D-Bus . . . 13

7 Daemon de liaison RFCOMM : `nav-bind-daemon.sh` 13
7.1 Rôle et perimetre . . . 13
7.2 Variables de configuration . . . 13
7.3 Logique de binding detaillee . . . 14
7.3.1 etape 1 : lecture de l'etat courant de rfcomm1 . . . 14
7.3.2 etape 2 : consultation SDP du telephone . . . 14
7.3.3 etape 3 : decision de rebinding . . . 14
7.3.4 etape 4 : rebinding . . . 14
7.4 Installation comme service systemd . . . 15
7.5 Diagnostic et depannage . . . 16

8 Application Android `NavRelay` 16
8.1 Structure du projet . . . 16
8.2 Compiler et installer . . . 16
8.3 Permissions et manifeste . . . 16
8.4 `MainActivity` : demarrage et ecran d'etat . . . 17
8.5 `NavServer` : serveur Bluetooth RFCOMM . . . 17
8.5.1 UUID et nom de service . . . 17
8.5.2 Cycle de vie du serveur . . . 17
8.5.3 Envoi des messages JSON . . . 18
8.5.4 Format JSON . . . 18
8.6 `MapsNotificationListener` . . . 18
8.6.1 Interception des notifications . . . 18
8.6.2 Lecture des textes : `getCharSequence()` . . . 19
8.6.3 Classement : ETA, distance, action . . . 19
8.6.4 Extraction de la distance . . . 19
8.6.5 Liaison du service et debogage . . . 20
8.6.6 Donnees GPS . . . 20
8.6.7 Verifier la connexion depuis le Pi . . . 20

9 Application LVGL (`lv_port_vscode/src`) 20
9.1 Macros de configuration . . . 20
9.2 Structures de donnees partagees . . . 20
9.3 Thread de navigation : `nav_thread_fn()` . . . 21
9.3.1 Connexion et reconnexion à `/dev/rfcomm1` . . . 21
9.3.2 Parsing JSON . . . 22
9.4 Thread musique : `music_thread_fn()` . . . 22
9.4.1 Connexion D-Bus et decouverte du lecteur . . . 22
9.4.2 Lecture des proprietes . . . 22
9.4.3 Commandes de contrôle . . . 22
9.5 Thread appels : `calls_thread_fn()` . . . 23
9.5.1 Decouverte du modem HFP . . . 23
9.5.2 Polling des appels . . . 23
9.5.3 Decrocher et raccrocher . . . 23
9.6 Timer UI (100 ms) . . . 23
9.6.1 Accents et polices LVGL . . . 24
9.7 Layout LVGL . . . 24
9.8 Callbacks des boutons . . . 24

10 Compteur de vitesse LVGL : `speedo_interactive.c/.h` 24
10.1 API publique . . . 24
10.2 Parametres du cadran . . . 25
10.3 Conversion vitesse/angle et aiguille . . . 25

11 Recuperation de jaquette : `album_art.c` 25
11.1 Signature . . . 25
11.2 Flux d'execution . . . 25

12 Ordre de demarrage complet 26

13 Verification et validation pre-lancement 26
13.1 Checklist systeme (apres SETUP.sh) . . . 26
13.2 Checklist appairage . . . 26
13.3 Checklist application . . . 27

14 Depannage general 27
14.1 « Waiting for route... » en permanence . . . 27
14.2 Distance « -- m » ou action incorrecte . . . 27
14.3 « No MediaPlayer » . . . 27
14.4 Aucun appel detecte . . . 27
14.5 La jaquette ne s'affiche pas . . . 27
14.6 Pair Button ne fonctionne pas . . . 28
14.7 Une fenetre Bluetooth du bureau apparait encore . . . 28
14.8 Le dashboard ne se ferme pas . . . 28
14.9 Commandes de logs utiles . . . 28

15 Limitations connues 28

16 Fichiers du projet 29
16.1 Fichiers essentiels . . . 29

1 DÉMARRAGE RAPIDE (NOUVELLE INSTALLATION)

1.1 Installation en 3 etapes

Pour une nouvelle installation de Raspberry Pi OS Bookworm, executer :

1. **Cloner le depôt et les sous-modules**

```
git clone --recursive \
    https://github.com/EEmeka33/Mini_Dashboard.git
cd Mini_Dashboard
```

Note : L'option `-recursive` remplace les deux commandes separees `git submodule update -init -recursive`, simplifiant le processus.

2. **Installation et configuration automatisée (SETUP.sh)**

```
bash SETUP.sh
```

Duree : 5-10 minutes. Installe paquets, configure services systemd et le daemon RFCOMM.

Le script installe et active automatiquement :

— `bluetooth.service` pour BlueZ (avec un `main.conf` valide pour BlueZ 5)
— `ofono.service` pour les appels HFP
— `nav-bind.service` pour le daemon RFCOMM
— `pulseaudio` pour la musique Bluetooth
— l'ajout de `pi5` aux groupes `bluetooth` et `dialout`
— le retrait du widget Bluetooth de la barre des taches (voir section 5.5)

Important : se deconnecter puis se reconnecter (ou redemarrer) une fois apres `SETUP.sh`, pour que les nouveaux groupes soient pris en compte.

3. **Compiler le dashboard**

```
cd lv_port_pc_vscode
rm -rf build
mkdir build
cd build
cmake ..
make -j4
```

4. **Vérifier que les services système tournent**

```
sudo systemctl status bluetooth.service
sudo systemctl status ofono.service
sudo systemctl status nav-bind.service
```

Tous les trois doivent montrer `active (running)`.

5. **Lancer le fichier compilé :**

```
cd lv_port_pc_vscode/bin/
./main
```

6. **Quitter le dashboard** : touche **Échap** (ou **Ctrl**+**Q**) dans l'application, **Ctrl**+**C** dans le terminal, ou `systemctl --user stop dashboard`. Le processus se termine proprement et rend l'ecran au bureau.

2 Nouveautés de la version 4

Problème (v3)	**Correction (v4)**
Le programme ne se fermait pas (Ctrl+C, fermeture fenetre)	SDL interceptait Ctrl+C et le pilote LVGL patche n'appelait plus `exit()`. `main.c` gere desormais SIGINT/SIGTERM, `SDL_QUIT` et Échap, puis arrete les threads proprement (section 4.7).
Il fallait relancer le dashboard apres le premier appairage	Le lecteur `MediaPlayer1` et le modem oFono n'etaient cherches qu'une fois. Ils sont maintenant recherches toutes les 3 s et apres chaque perte.
L'UI pouvait geler plusieurs secondes	Plus aucun appel D-Bus ou reseau sous le verrou partage ni dans le thread UI ; connexion D-Bus privee par thread, timeout 3 s ; jaquette telechargee dans un thread dedie.
Popup d'appairage du bureau Pi OS (sortie du plein ecran)	Agent BlueZ integre `bt_agent.c` (agent par defaut, acceptation automatique pendant la fenetre Pair) et retrait du widget Bluetooth de la barre des taches.
Distance toujours « 0 m »	NavRelay lisait le titre Maps avec `getString()` (renvoie `null` pour un texte formate) et ne reconnaissait pas l'espace insecable de « 300 m ».
L'heure d'arrivee affichee comme action	Les lignes d'ETA sont reconnues et separees ; l'ETA s'affiche sous l'instruction.
La navigation s'arretait apres une reconnexion du Pi	`NavServer` s'arretait definitivement a la premiere erreur d'ecriture ; il continue maintenant d'accepter les connexions.
Accents absents a l'ecran	Les polices Montserrat de LVGL sont ASCII : les accents sont convertis (« à » → « a ») avant affichage.

3 Vue d'ensemble du système

Le projet Mini Dashboard est un tableau de bord automobile embarqué tournant sur Raspberry Pi 5 Sous un écran circulaire 1080 × 1080 pixels. Il agrège trois flux de données en temps réel :

- **Navigation** : données de Google Maps relayées depuis un smartphone Android via Bluetooth RFCOMM (profil SPP), lues sur `/dev/rfcomm1` sous forme de lignes JSON.
- **Musique Bluetooth** : métadonnées A2DP/AVRCP et jaquette, obtenues par D-Bus via `org.bluez.MediaPlayer1`.
- **Appels HFP** : détection et gestion des appels entrants via oFono et `org.ofono.VoiceCallManager`.

3.1 Architecture globale

Smartphone Android	Couche systeme Pi	Application LVGL
NavRelay (Kotlin) Google Maps notifications	BlueZ + RFCOMM `/dev/rfcomm1`	`nav_thread_fn()` JSON → `nav_info_t`
Lecteur audio A2DP/AVRCP	BlueZ `MediaPlayer1` D-Bus systeme	`music_thread_fn()` `get_music_properties()`
Telephonie Android Profil HFP	BlueZ + oFono D-Bus systeme	`calls_thread_fn()` `org.ofono.VoiceCall`

3.2 Prérequis matériels et logiciels

- Raspberry Pi 5 avec Raspberry Pi OS Bookworm (64-bit recommande)
- ecran 1080 × 1080, connecte via HDMI ou DSI
- Contrôleur Bluetooth integre ou USB (`hci0`)
- Smartphone Android avec l'application **NavRelay** installee et Google Maps

3.2.1 Build tools (obligatoire pour compiler)

```
sudo apt update && sudo apt install -y \
  build-essential cmake git pkg-config
```

3.2.2 Paquets intime et dépendances

```
sudo apt update
sudo apt install -y pulseaudio pulseaudio-module-bluetooth
sudo apt install -y bluez bluez-tools ofono modem-manager
sudo apt install -y libdbus-1-dev libcurl4-openssl-dev libjson-c-dev
sudo apt install -y libsdl2-dev libsdl2-image-dev
sudo apt install -y alsa-utils dbus
```

Ou en une seule commande (identique à SETUP.sh) :

```
sudo apt update
sudo apt install -y pulseaudio pulseaudio-module-bluetooth bluez
sudo apt install -y bluez-tools ofono modem-manager alsa-utils dbus
sudo apt install -y libdbus-1-dev libcurl4-openssl-dev libjson-c-dev
sudo apt install -y libsdl2-dev libsdl2-image-dev
```

3.2.3 Installation automatisée (RECOMMANDÉE)

Tous les paquets et configurations ci-dessus sont appliqués automatiquement :

```
bash /home/pi5/dashboard/SETUP.sh
```

Ce script prend 5-10 minutes et configure également les services systemd dans les bonnes conditions.

4 Compilation et mise en place du projet LVGL

4.1 Cloner le depôt et initialiser les sous-modules

Le projet utilise LVGL comme sous-module Git. L'option `-recursive` initialise automatiquement les sous-modules lors du clonage :

```
git clone --recursive https://github.com/EEmeka33/Mini_Dashboard.git
cd Mini_Dashboard
```

Avantage : Une seule commande `git clone -recursive` remplace les deux commandes separees `git clone` puis `git submodule update -init -recursive`.

4.2 Structure du repertoire `lv_port_vscode`

```
lv_port_vscode/
  src/
    hal/
        hal.c / hal.h       # Pilote SDL2 pour LVGL
    main.c                  # Point d'entree, UI, threads
    speedo_interactive.c    # Widget compteur de vitesse
    speedo_interactive.h    # API publique du compteur
    album_art.c             # Recuperation jaquette iTunes
    bt_agent.c / bt_agent.h # Agent d'appairage BlueZ (bouton Pair)
  lvgl/                     # Sources LVGL (sous-module)
  lv_conf.h                 # Configuration LVGL
  CMakeLists.txt
  build/                    # Repertoire de compilation
```

4.3 Configuration LVGL (`lv_conf.h`)

Les polices suivantes doivent être activees car elles sont utilisees dans `main.c` et `speedo_interactive.c` :

Listing 1 – lv_conf.h — polices a activer

```
#define LV_FONT_MONTSERRAT_14 1
#define LV_FONT_MONTSERRAT_16 1
#define LV_FONT_MONTSERRAT_18 1
#define LV_FONT_MONTSERRAT_22 1
#define LV_FONT_MONTSERRAT_24 1
#define LV_FONT_MONTSERRAT_48 1
```

Le pilote systeme de fichiers POSIX doit être active pour le chargement d'images JPEG :

```
#define LV_USE_FS_POSIX 1
#define LV_FS_POSIX_LETTER 'A'
```

Le decodeur JPEG integre doit être active pour que `lv_image_set_src()` charge `cover.jpg` :

```
#define LV_USE_TJPGD 1 /* ou LV_USE_LIBJPEG_TURBO selon la version */
```

Ligne pour afficher l'application en Fullscreen :

```
#define LV_SDL_FULLSCREEN 1
```

4.4 Compilation complete avec CMake (RECOMMANDÉE)

```
cd lv_port_vscode
rm -rf build
mkdir build
cd build
cmake ..
make -j4
```

Ou en une seule ligne :

```
cd lv_port_vscode && rm -rf build && mkdir build && cd build && cmake .. && make -j4
```

L'exécutable sera créé à `bin/main`.

4.5 Compilation manuelle (alternative)

```
cd lv_port_vscode
gcc -O2 -o dashboard \
  src/main.c src/speedo_interactive.c src/album_art.c src/bt_agent.c \
  src/mouse_cursor_icon.c src/hal/hal.c \
  $(find lvgl/src -name "*.c" | tr '\n' ' ') \
  $(pkg-config --cflags --libs dbus-1 libcurl json-c sdl2) \
  -I. -Ilvgl -Ilvgl/src \
  -lm -lpthread
```

4.6 Lancer le dashboard

Le programme necessite une session graphique (SDL2) :

```
export DISPLAY=:0
cd lv_port_vscode/bin
./main
```

Pour un lancement automatique au demarrage, un service systemd peut être configure via SETUP.sh.

4.7 Arrêt propre du dashboard

En v3, le pilote SDL de LVGL avait ete modifie pour ne plus appeler `exit()` sur `SDL_QUIT`, mais `main()` tournait en `while(1)` sans jamais le detecter. De plus, SDL installe ses propres gestionnaires de signaux et transforme Ctrl+C en evenement `SDL_QUIT`. Resultat : le processus ne s'arretait jamais.

La v4 procede ainsi :

1. `SDL_SetHint(SDL_HINT_NO_SIGNAL_HANDLERS, "1")` avant l'initialisation de SDL, puis gestionnaires `sigaction` pour SIGINT, SIGTERM et SIGHUP qui positionnent le drapeau `g_quit`.
2. Un filtre d'evenements SDL (`SDL_SetEventFilter`) intercepte `SDL_QUIT`, la fermeture de fenetre, Échap et Ctrl+Q. Ces evenements sont supprimes pour que LVGL ne detruise pas l'affichage pendant qu'un timer l'utilise.
3. La boucle principale tourne tant que `app_running()` est vrai.
4. À la sortie : arret de l'agent Bluetooth (desenregistrement, Pi rendu invisible), reveil des threads par `SIGUSR1` (interrompt un `open()` ou `poll()` bloquant), `pthread_timedjoin_np()` avec 3 s maximum par thread, puis `lv_sdl_quit()`.
5. Filet de securite : `alarm(6)` termine le processus si une etape bloque.

5 Pair Button et agent d'appairage Bluetooth

5.1 Principe : un agent BlueZ integre au dashboard

Quand un telephone demande a s'appairer, BlueZ interroge l'*agent par defaut* (`org.bluez.Agent1`) pour obtenir une confirmation. En v3, cet agent etait celui du bureau Pi OS, qui affichait une fenetre de confirmation et faisait sortir le dashboard du plein ecran.

En v4, le dashboard enregistre son propre agent (`src/bt_agent.c`) et le declare agent par defaut via `AgentManager1.RequestDefaultAgent`. BlueZ lui envoie donc toutes les demandes, auxquelles il repond sans aucune fenetre. L'agent reaffirme sa place toutes les 15 s, au cas ou le bureau la reprendrait, et se desenregistre a la fermeture.

Listing 2 – bt_agent.h — API publique (non bloquante)

```c
#define BT_PAIR_WINDOW_S 180

int bt_agent_start(void); /* lance le thread agent */
void bt_agent_stop(void); /* desenregistre et arrete */
void bt_agent_open_pairing_window(void); /* appele par le bouton Pair */
void bt_agent_get_status(char *buf, size_t len); /* texte sous Pair */
```

Tout le travail D-Bus est fait dans un thread dedie ; l'UI ne fait que poser une demande. Ce module remplace `bt_pairing.h` et l'appel a `bt_make_discoverable.sh`.

5.2 Fenetre d'appairage (bouton Pair)

Un appui sur **Pair** :

1. met l'adaptateur sous tension (`Powered`) ;
2. regle `DiscoverableTimeout` et `PairableTimeout` a 180 s ;
3. active `Pairable` et `Discoverable` sur `org.bluez.Adapter1` ;
4. affiche le compte a rebours sous le bouton (« Visible 2 :59 - pair from your phone »).

Pendant la fenetre, l'agent verifie chaque seconde les appareils BlueZ. Des qu'un telephone devient `Paired`, il est marque `Trusted` (reconnexions futures sans autorisation), une connexion est lancee (`Device1.Connect`), la fenetre se ferme et le dashboard affiche « Paired : <nom> ».

5.3 Regles d'acceptation

Requete BlueZ	Fenetre ouverte	Fenetre fermee
`RequestConfirmation`	acceptee	refusee
`RequestAuthorization`	acceptee	refusee
`RequestPinCode`	`"0000"`	refusee
`RequestPasskey`	`0`	refusee
`AuthorizeService`	acceptee	acceptee si deja appaire

Securite : hors de la fenetre de 3 minutes, aucun nouvel appareil ne peut s'appairer. Seuls les telephones deja appaires peuvent se reconnecter.

5.4 Parametres (`bt_agent.c`)

Listing 3 – bt_agent.c — configuration

```c
#define AGENT_PATH "/org/minidashboard/agent"
#define AGENT_CAPABILITY "NoInputNoOutput"
#define BT_ALIAS NULL /* ex: "Mini Dashboard" */
#define DBUS_TIMEOUT_MS 3000
```

`NoInputNoOutput` Appairage *Just Works* : le telephone demande seulement « Associer ? ».

`DisplayYesNo` Comparaison de code : le code s'affiche sur le telephone, le Pi le confirme automatiquement.

`BT_ALIAS` Nom visible depuis le telephone ; `NULL` conserve le nom du Pi (« raspberrypi »).

5.5 Popup « Pairing successful » du bureau Pi OS

Meme avec l'agent integre, le **widget Bluetooth de la barre des taches** affichait encore une fenetre « Pairing successful / Connection successful ». Ce widget (`lxplug-bluetooth`) surveille tous les appareils BlueZ et ouvre sa propre fenetre GTK des qu'un appareil passe a `Paired`, quel que soit l'agent qui a accepte. Le dashboard ne peut donc pas l'empecher : il faut retirer le widget.

Le script `disable_panel_bluetooth.sh`, appele par `SETUP.sh`, le retire pour les deux bureaux de Pi OS :

- **Wayland** (`wf-panel-pi`) : supprime `bluetooth` de la cle `widgets_right` dans le fichier `~/.config/wf-panel-pi/wf-panel-pi.ini` ;
- **X11** (`lxpanel-pi`) : supprime le bloc `Plugin { type=bluetooth }` du fichier `~/.config/lxpanel-pi/panels/panel`.

```
bash disable_panel_bluetooth.sh # retire le widget (sans sudo, en pi5)
bash disable_panel_bluetooth.sh --restore # remet la configuration d'origine
```

Le Bluetooth continue de fonctionner (`bluetoothd` est un service systeme) ; seule l'icone de la barre des taches disparait.

5.6 Appairage initial du telephone

5.6.1 Methode 1 : Via le bouton Pair du dashboard (RECOMMANDÉE)

1. **Lancer le dashboard** : `./main` depuis `lv_port_pc_vscode/bin`
2. **Cliquer sur le bouton bleu « Pair »** : le Pi devient visible 3 minutes
3. **Sur le telephone** : Parametres → Bluetooth → Associer un appareil
4. **Selectionner « raspberrypi »** et accepter **sur le telephone uniquement**
5. Le dashboard affiche « Paired : <nom du telephone> » ; aucune fenetre ne s'ouvre sur le Pi

Avantage : aucune ligne de commande, aucune confirmation sur le Pi, le plein ecran est conserve. Le telephone est automatiquement `Trusted` et son adresse MAC est detectee par `main.c`.

5.6.2 Methode 2 : En ligne de commande (manuel)

```
sudo bluetoothctl
[bluetooth]# power on
[bluetooth]# agent on
[bluetooth]# default-agent
[bluetooth]# scan on
```

Attendre que le telephone apparaisse :

```
[NEW] Device 2C:BE:EB:7F:2D:FA Pixel 7
```

Continuer l'appairage :

```
[bluetooth]# scan off
[bluetooth]# pair 2C:BE:EB:7F:2D:FA
```

Confirmer le PIN sur le telephone (ou pas si le Pi envoie le PIN automatiquement).

```
[bluetooth]# trust 2C:BE:EB:7F:2D:FA
[bluetooth]# connect 2C:BE:EB:7F:2D:FA
[bluetooth]# quit
```

Note : `bluetoothctl default-agent` prend la place de l'agent du dashboard jusqu'a la prochaine reaffirmation (15 s maximum). Apres appairage, l'adresse MAC est sauvegardee par `main.c` dans `/tmp/connected_phone_mac.txt`.

6 Configuration systeme Bluetooth

6.1 Configuration BlueZ pour la musique et les appels

6.1.1 Fichier `main.conf` ecrit par `SETUP.sh`

La version precedente de `SETUP.sh` ecrivait des cles que BlueZ 5 ne connait pas (`Enable=Source`, `HandsfreeDevice`, `Trustable`) et qui etaient ignorees. La v4 ecrit uniquement des cles valides (l'ancien fichier est sauvegarde une seule fois dans `main.conf.orig`) :

Listing 4 – /etc/bluetooth/main.conf (v4)

```
[General]
# Classe "Car audio" : le telephone voit le Pi comme un autoradio
Class = 0x200420
# Fenetre d'appairage ouverte par le bouton Pair (secondes)
DiscoverableTimeout = 180
PairableTimeout = 0
# Re-appairage sans confirmation si le telephone a "oublie" le Pi
JustWorksRepairing = always
FastConnectable = true

[Policy]
# Adaptateur allume au demarrage + reconnexion auto au telephone connu
AutoEnable = true
ReconnectAttempts = 7
ReconnectIntervals = 1,2,4,8,16,32,64
```

Note sur `Disable=Headset` : cette directive vient de BlueZ 4 (`audio.conf`) et n'est pas reconnue par BlueZ 5. Sous Bookworm, c'est la configuration PipeWire de la section 6.2 qui confie le HFP a oFono.

6.1.2 Groupes de l'utilisateur

L'agent d'appairage et la lecture de `/dev/rfcomm1` necessitent :

```
sudo usermod -aG bluetooth,dialout pi5 # fait par SETUP.sh
# puis se deconnecter / se reconnecter
```

6.1.3 Activation du service oFono

```
sudo systemctl enable ofono
sudo systemctl start ofono
sudo systemctl status ofono
```

6.1.4 Redemarrage des services Bluetooth

Apres toute modification de `main.conf` :

```
sudo systemctl restart bluetooth
sudo systemctl restart ofono
```

6.2 Configuration PipeWire pour HFP via oFono

Sur Raspberry Pi OS Bookworm, PipeWire remplace PulseAudio. Pour que les appels HFP passent par oFono :

```
sudo mkdir -p /etc/pipewire/pipewire-pulse.conf.d/
sudo nano /etc/pipewire/pipewire-pulse.conf.d/10-ofono.conf
```

Listing 5 – 10-ofono.conf

```
pulse.properties = {
  bluez5.headset-roles = [ hfp_hf hsp_hs ]
  bluez5.hfphsp-backend = ofono
}
```

Redemarrer PipeWire dans la session utilisateur (sans `sudo`) :

```
systemctl --user restart pipewire pipewire-pulse wireplumber
```

6.3 Verification de la pile D-Bus

Apres connexion du telephone et lancement d'un morceau :

```
# Verifier la presence d'un objet MediaPlayer1 (musique)
busctl tree org.bluez
# Attendu : /org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA/player0

# Inspecter les proprietes
busctl introspect org.bluez \
  /org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA/player0

# Verifier oFono (appels HFP)
busctl call org.ofono / org.ofono.Manager GetModems
# Doit lister un modem avec org.ofono.VoiceCallManager
```

7 Daemon de liaison RFCOMM : `nav-bind-daemon.sh`

7.1 Rôle et perimetre

`nav-bind-daemon.sh` est le seul demon specifique au projet côte Raspberry Pi. Il maintient en permanence le binding entre `/dev/rfcomm1` et le service `NavJsonService` de l'application Android NavRelay.

Le numero de canal RFCOMM assigne dynamiquement par Android peut varier à chaque reconnexion ; il ne peut être connu qu'en consultant le registre SDP du telephone à la volee, d'où la necessite de ce demon.

7.2 Variables de configuration

A partir de la version amelioree du daemon, le MAC du telephone est detecte automatiquement par `main.c` et ecrit dans le fichier `/tmp/connected_phone_mac.txt`.

Listing 6 – nav-bind-daemon.sh — detection auto du MAC

```
MAC_FILE="/tmp/connected_phone_mac.txt"
if [ -f "$MAC_FILE" ]; then
  PHONE_MAC=$(cat "$MAC_FILE" 2>/dev/null)
else
```

```
  PHONE_MAC="2C:BE:EB:7F:2D:FA" # Fallback si fichier absent
fi
TARGET_SERVICE_NAME="NavJsonService"
```

Avantage : Plus besoin de modifier manuellement le MAC. Le daemon se reconfigure automatiquement lors du reconnexion d'un telephone different.

Comment ca marche :

1. `main.c` detecte le telephone connecte via D-Bus (org.bluez.Device1).
2. `main.c` extrait l'adresse MAC et l'ecrit dans `/tmp/connected_phone_mac.txt` (verification toutes les 5 s ; ecriture uniquement si l'adresse change, via un fichier temporaire puis `rename()` pour que le daemon ne lise jamais un fichier incomplet).
3. Le daemon relance la lecture du fichier chaque cycle (5 secondes).
4. Si le MAC change (nouveau telephone), le daemon se reconfigure automatiquement.

7.3 Logique de binding detaillee

Le script tourne dans une boucle infinie avec 5 secondes d'attente entre chaque cycle.

7.3.1 etape 1 : lecture de l'etat courant de rfcomm1

```
CURRENT_LINE=$(rfcomm | awk -v mac="$PHONE_MAC" \
  '$1=="rfcomm1:" && $2==mac {print $0}')
```

La sortie de `rfcomm` ressemble à :

```
rfcomm1: 2C:BE:EB:7F:2D:FA channel 7 config
```

Si aucune ligne ne correspond, `CURRENT_LINE` est vide et un rebind sera declenche.

7.3.2 etape 2 : consultation SDP du telephone

```
SDP_OUTPUT=$(sdptool browse "$PHONE_MAC" 2>/dev/null || true)

TARGET_CHANNEL=$(printf '%s\n' "$SDP_OUTPUT" \
  | awk -v name="$TARGET_SERVICE_NAME" '
    $0 ~ "Service Name: "name { in_block=1 }
    in_block && /Channel:/ { print $2; exit }
  ')
```

`sdptool browse` parcourt tous les services Bluetooth du telephone. Si `TARGET_CHANNEL` est vide (NavRelay non demarre ou telephone hors portee), le cycle recommence apres 5 s.

7.3.3 etape 3 : decision de rebinding

Trois cas declenchent un rebind :

— `rfcomm1` n'existe pas ou n'est pas associe à `PHONE_MAC`.
— `rfcomm1` existe mais est en etat `closed`.
— `rfcomm1` existe mais pointe sur un canal different de `TARGET_CHANNEL`.

7.3.4 etape 4 : rebinding

```
rfcomm release 1 2>/dev/null || true
rfcomm bind 1 "$PHONE_MAC" "$TARGET_CHANNEL"
```

Apres un bind reussi, `/dev/rfcomm1` est disponible et `nav_thread_fn()` peut l'ouvrir.

7.4 Installation comme service systemd

```
sudo nano /etc/systemd/system/nav-bind.service
```

Listing 7 – /etc/systemd/system/nav-bind.service

```
[Unit]
Description=RFCOMM binding daemon for NavRelay navigation
After=bluetooth.target
Wants=bluetooth.target

[Service]
Type=simple
ExecStart=/bin/bash /home/pi5/dashboard/nav-bind-daemon.sh
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

```
# 1. Copier le script du daemon
sudo cp nav-bind-daemon.sh /usr/local/bin/nav-bind-daemon.sh

# 2. Rendre executable
sudo chmod +x /usr/local/bin/nav-bind-daemon.sh

# 3. Creer le service systemd
sudo tee /etc/systemd/system/nav-bind.service > /dev/null << 'EOF'
[Unit]
Description=RFCOMM auto-bind daemon for NavJsonService
After=bluetooth.target
Wants=bluetooth.target

[Service]
Type=simple
ExecStart=/usr/local/bin/nav-bind-daemon.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

# 4. Recharger systemd
sudo systemctl daemon-reload

# 5. Activer au demarrage
sudo systemctl enable nav-bind.service

# 6. Demarrer maintenant
sudo systemctl start nav-bind.service

# 7. Verifier
systemctl status nav-bind.service
```

7.5 Diagnostic et depannage

`sdptool returned nothing` Le telephone n'est pas connecte. Verifier : `bluetoothctl info 2C:BE:EB:7F:2D:FA` (champ `Connected: yes`).

`cannot find service "NavJsonService"` NavRelay n'est pas demarre. Ouvrir NavRelay et appuyer sur « Start ».

`rfcomm bind failed` Module noyau manquant : `sudo modprobe rfcomm`. Verifier aussi que le service tourne en root.

`/dev/rfcomm1` **introuvable apres binding** Ajouter l'utilisateur au groupe `dialout` : `sudo usermod -aG dialout pi5`, puis se reconnecter.

rfcomm1 se rebind en boucle Qualite de signal Bluetooth insuffisante ou telephone non marque comme de confiance (`trust` dans `bluetoothctl`).

8 Application Android `NavRelay`

8.1 Structure du projet

```
NavRelay/
  app/src/main/
    AndroidManifest.xml
    java/com/example/navrelay/
      MainActivity.kt
      NavServer.kt
      MapsNotificationListener.kt
    res/layout/
      activity_main.xml
```

8.2 Compiler et installer

1. Ouvrir Android Studio et importer le projet `NavRelay/`.
2. Verifier que `compileSdkVersion` ≥ 33.
3. Connecter le telephone en USB avec le debogage active.
4. Cliquer sur *Run* ou : `./gradlew installDebug`

8.3 Permissions et manifeste

— `BLUETOOTH`, `BLUETOOTH_ADMIN`, `BLUETOOTH_CONNECT`, `BLUETOOTH_ADVERTISE`, `BLUETOOTH_SCAN`
— `ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`
— `POST_NOTIFICATIONS` (Android 13+)
— Service `MapsNotificationListener` declare avec l'action `BIND_NOTIFICATION_LISTENER_SERVICE`

IMPORTANT — etape obligatoire apres installation : Accorder l'acces aux notifications manuellement apres le premier lancement :

1. Ouvrir *Parametres* → *Applications*
2. Chercher *Acces speciaux* ou *Permissions avancees*
3. Selectionner *Acces aux notifications*
4. Activer `NavRelay`

`MainActivity` ouvre automatiquement cet ecran via `Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS` si l'acces n'est pas encore accorde.

Apres chaque reinstallation (*Run* depuis Android Studio), l'application est tuee : rouvrir NavRelay et appuyer a nouveau sur **Start**. Android ne relie parfois pas le service de notifications

apres une mise a jour ; l'ecran d'etat l'indique (« NOT bound ») et il suffit alors de desactiver puis reactiver NavRelay dans *Acces aux notifications*.

Note : Sur certaines ROM Android, desactiver l'optimisation de batterie pour NavRelay afin d'eviter que le service soit tue en arriere-plan.

8.4 `MainActivity` : demarrage et ecran d'etat

Listing 8 – MainActivity.kt — bouton Start

```
startButton.setOnClickListener {
    NavServer.start(this)
    if (hasNotificationAccess()) {
        // Apres une reinstallation, Android ne relie souvent pas
        // le service de notifications : on le force.
        NotificationListenerService.requestRebind(
            ComponentName(this, MapsNotificationListener::class.java))
    } else {
        startActivity(Intent(Settings.ACTION_NOTIFICATION_LISTENER_SETTINGS))
    }
}
```

L'ecran est rafraichi toutes les secondes et indique l'etat de chaque maillon :

Ligne	Signification
1. Notification access	`OK` ou `MISSING` (permission non accordee)
2. Maps listener	`bound` ou `NOT bound` (desactiver/reactiver l'acces)
3. Bluetooth server	`Waiting for the Pi...`, `Connected to ...`, `Bluetooth is OFF`
4. Messages sent to Pi	compteur des lignes JSON envoyees
Last Maps info	derniere action, distance et ETA reconnues

Les permissions Bluetooth et localisation sont demandees au lancement via `ActivityCompat.requestPermiss` avec les variantes `BLUETOOTH_CONNECT / ADVERTISE / SCAN` pour Android 12+.

8.5 `NavServer` : serveur Bluetooth RFCOMM

8.5.1 UUID et nom de service

Listing 9 – NavServer.kt — constantes

```
private const val SERVICE_NAME = "NavJsonService"
val SPP_UUID: UUID =
    UUID.fromString("00001101-0000-1000-8000-00805F9B34FB")
```

L'UUID `00001101-...` est l'UUID standardise du *Serial Port Profile* (SPP). `NavJsonService` est le nom que `nav-bind-daemon.sh` recherche dans le registre SDP. Ces deux valeurs doivent rester coherentes.

8.5.2 Cycle de vie du serveur

`start()` lance un thread qui :

1. Recupere l'adaptateur Bluetooth ; s'il est eteint, l'etat affiche « Bluetooth is OFF » et le thread reessaie toutes les 2 s.

2. Appelle `listenUsingRfcommWithServiceRecord(SERVICE_NAME, SPP_UUID)`, enregistrant le service dans le registre SDP avec un canal assigne dynamiquement. C'est ce canal que le daemon decouvre via `sdptool`.
3. Boucle sur `accept()` pour accepter la connexion du Raspberry Pi. À chaque reconnexion, l'ancienne socket et son flux sont fermes et remplaces.
4. En cas d'erreur du serveur, il est recree automatiquement apres 2 s.

8.5.3 Envoi des messages JSON

Listing 10 – NavServer.kt — send()

```
fun send(json: JSONObject) {
    val out = outRef.get() ?: return // Pi pas encore connecte
    try {
        val line = json.toString() + "\n"
        out.write(line.toByteArray(Charsets.UTF_8))
        out.flush()
        sentCount++
    } catch (e: Exception) {
        // Le Pi s'est deconnecte : on ferme SEULEMENT ce client,
        // le serveur continue d'accepter les connexions.
        dropClient()
        if (running) state = "Waiting␣for␣the␣Pi..."
    }
}
```

Chaque message est une ligne JSON terminee par `\n` en UTF-8.

Correction v4 : en v3, une erreur d'ecriture appelait `stop()`, ce qui arretait definitivement le serveur. Chaque redemarrage du dashboard (qui ferme `/dev/rfcomm1`) coupait donc la navigation jusqu'a un nouvel appui sur Start.

8.5.4 Format JSON

Listing 11 – Exemple de message JSON envoye par NavRelay

```
{
  "timestamp": 1711550400,
  "next_action": "Tournez a droite sur Rue de la Paix",
  "distance_to_next_m": 320,
  "eta": "12 min - 4,5 km - Arrivee a 14:32",
  "speed_kmh": 48.5,
  "lat": 48.8566,
  "lon": 2.3522
}
```

`distance_to_next_m` distance avant la prochaine manoeuvre, en metres ; `-1` si elle est inconnue (le dashboard affiche alors « -- m »).

`eta` ligne d'arrivee de Maps (nouveau en v4), affichee sous l'instruction.

8.6 `MapsNotificationListener`

8.6.1 Interception des notifications

`onNotificationPosted()` filtre les notifications dont le `packageName` est `"com.google.android.apps.maps"`. Une notification de navigation typique (FR) :

Champ	Exemple	Role
`EXTRA_TITLE`	« 300 m »	distance avant la manoeuvre
`EXTRA_TEXT`	« Tournez à droite sur Rue X »	prochaine action
`EXTRA_SUB_TEXT`	« 12 min · 4,5 km · Arrivée à 14 :32 »	ETA

La disposition varie selon la version de Maps (la distance peut etre dans la meme ligne que l'action : « 300 m · Tournez à droite »). Le listener ne se fie donc pas a la position des champs mais **classe le contenu**.

8.6.2 Lecture des textes : `getCharSequence()`

Maps place du texte formate (`SpannableString`) dans le titre. En v3, le titre etait lu avec `getString()`, qui renvoie `null` dans ce cas : la distance etait perdue (d'ou « 0 m ») et la ligne d'ETA etait prise comme action. La v4 lit tous les champs avec `getCharSequence()` :

```
private fun Bundle.text(key: String): String =
    getCharSequence(key)?.toString()?.trim().orEmpty()
```

Champs lus, par ordre de priorite : `EXTRA_TITLE`, `EXTRA_TITLE_BIG`, `EXTRA_TEXT`, `EXTRA_BIG_TEXT`, `EXTRA_TEXT_LINES`, `EXTRA_SUB_TEXT`, `EXTRA_INFO_TEXT`.

8.6.3 Classement : ETA, distance, action

1. **ETA** : une ligne est classee ETA si elle contient une heure (`14:32`, `14 h 32`), une duree (`12 min`, `1 h`) ou le mot « arrivée » / « ETA ». Elle n'est jamais utilisee comme action ni comme distance (sinon la distance totale « 4,5 km » remplacerait celle de la manoeuvre).
2. **Distance** : premiere distance trouvee dans les lignes non-ETA.
3. **Action** : premiere ligne non-ETA qui contient autre chose qu'une distance ; une distance en tete (« 300 m · ») est retiree.
4. **Filet de securite** : si rien n'est reconnu, la premiere ligne est envoyee telle quelle (comportement v3) plutot que de ne rien envoyer.

Listing 12 – MapsNotificationListener.kt — expressions regulieres

```
// Espaces normales + insecables (Maps FR : U+00A0 / U+202F)
private const val SP = """[\s\u00A0\u202F]"""

// (?!\p{L}) : "12 min" n'est pas pris pour "12 m"
private val DIST_RE = Regex(
    """(?<![\d.,])(\d+(?:[.,]\d+)?)$SP*(km|m|mi|ft|yd)(?!\p{L})""",
    RegexOption.IGNORE_CASE)

private val ETA_RE = Regex(
    """\b\d{1,2}$SP?[:h]$SP?\d{2}\b|\b\d+$SP*(min|h|hr|hrs)\b|arriv|\beta\b""",
    RegexOption.IGNORE_CASE)
```

8.6.4 Extraction de la distance

`parseDistanceMeters()` reconnait « `320 m` », « `1,2 km` », « `1.2 km` », ainsi que `mi`, `ft` et `yd`. Tout est converti en metres ; la fonction renvoie `-1` si aucune distance n'est trouvee.

8.6.5 Liaison du service et debogage

`onListenerConnected()` / `onListenerDisconnected()` tiennent a jour l'etat « bound » affiche dans `MainActivity` ; en cas de deconnexion, le service demande lui-meme a Android d'etre relie (`requestRebind`). Chaque notification Maps est journalisee :

```
adb logcat -s NavRelay
# title='300 m' text='Tournez a droite...' sub='12 min - ...'
# -> next='Tournez a droite...' dist=300 eta='12 min - ...'
```

8.6.6 Donnees GPS

`requestLocationUpdates()` demande des mises à jour GPS toutes les secondes (deplacement minimal 1 m). La vitesse `Location.speed` (m/s) est convertie en km/h (`speed * 3.6f`) dans `onLocationChanged()`, puis integree dans le prochain message JSON.

8.6.7 Verifier la connexion depuis le Pi

```
# Verifier que NavJsonService est visible en SDP
sdptool browse 2C:BE:EB:7F:2D:FA | grep -A5 "NavJsonService"
# Sortie attendue :
# Service Name: NavJsonService
# Channel: 7 (numero variable)

# Lire le flux JSON en direct
cat /dev/rfcomm1
```

9 Application LVGL (`lv_port_vscode/src`)

9.1 Macros de configuration

Listing 13 – main.c — macros de configuration

```
1 #define SCREEN_W 1080
2 #define SCREEN_H 1080
3 #define MUSIC_COVER_PATH_FS "/home/pi5/dashboard/cover.jpg"
4 #define MUSIC_COVER_PATH "A:/home/pi5/dashboard/cover.jpg"
5 #define CENTER_RADIUS 250
6 #define BOTTOM_PLAYER_HEIGHT 250
```

`MUSIC_COVER_PATH_FS` est le chemin POSIX reel pour ecrire et verifier la jaquette. `MUSIC_COVER_PATH` utilise le prefixe `A:` du pilote LVGL POSIX initialise par `lv_fs_posix_init()`.

9.2 Structures de donnees partagees

Toutes les donnees partagees entre les threads et le timer UI sont stockees dans l'instance globale `g_state` de type `app_state_t`, protegee par `g_state.lock` (mutex POSIX).

Listing 14 – main.c — structures de donnees (v4)

```
1 typedef struct {
2     char instruction[256]; /* Instruction de virage courante */
3     double distance; /* Distance en metres (-1 = inconnue) */
4     double next_distance;
5     char street_name[256]; /* Nom de la rue courante */
6     char eta[128]; /* "12 min - 4,5 km - Arrivee 14:32" */
```

```
    double speed; /* Vitesse vehicule en km/h */
    double lat;
    double lon;
} nav_info_t;

typedef struct {
    char title[256];
    char artist[256];
    char album[256];
    char status[32]; /* "playing", "paused", etc. */
    unsigned int position_ms;
    unsigned int duration_ms;
} music_info_t;

typedef struct {
    char state[32]; /* "active","incoming","dialing"... */
    char line_id[64]; /* Numero de telephone */
    char name[64]; /* Nom du contact */
    char call_path[256]; /* Chemin D-Bus de l'appel oFono */
} call_info_t;

/* Commandes posees par l'UI, executees par les threads */
typedef enum { MEDIA_NONE, MEDIA_PLAY, MEDIA_PAUSE, MEDIA_NEXT, MEDIA_PREV }
    media_cmd_t;
typedef enum { CALL_NONE, CALL_ANSWER, CALL_HANGUP } call_cmd_t;

typedef struct {
    nav_info_t nav;
    int nav_connected;
    music_info_t music;
    unsigned int cover_version; /* +1 a chaque nouvelle jaquette */
    call_info_t current_call;
    int has_incoming_call;
    media_cmd_t media_cmd;
    call_cmd_t call_cmd;
    char call_cmd_path[256];
    pthread_mutex_t lock;
} app_state_t;
```

Regle de verrouillage (v4) : le verrou `g_state.lock` ne sert qu'a copier des donnees. Aucun appel D-Bus, reseau ou LVGL n'est fait en le tenant. En v3, le thread musique gardait le verrou pendant les appels D-Bus (timeout par defaut 25 s) et le telechargement iTunes, ce qui figeait le timer UI.

Connexions D-Bus : chaque thread ouvre sa propre connexion privee (`dbus_bus_get_private()`, `exit_on_disconnect` desactive) et tous les appels ont un timeout de 3 s (`DBUS_TIMEOUT_MS`). En v3, les deux threads ecrasaient la meme variable globale `dbus_conn`.

9.3 Thread de navigation : `nav_thread_fn()`

9.3.1 Connexion et reconnexion à `/dev/rfcomm1`

Listing 15 – main.c — boucle de lecture rfcomm1 (v4)

```
while (app_running()) {
    if (fd < 0) {
        /* bloque jusqu'a la connexion ; SIGUSR1 l'interrompt a l'arret */
        fd = open("/dev/rfcomm1", O_RDONLY | O_NOCTTY);
        if (fd < 0) { sleep_ms_interruptible(2000, NULL); continue; }
```

```
        fcntl(fd, F_SETFL, fcntl(fd, F_GETFL) | O_NONBLOCK);
    }
    struct pollfd pfd = { .fd = fd, .events = POLLIN };
    int pr = poll(&pfd, 1, 500); /* reveil periodique pour l'arret */
    if (pr == 0 || (pr < 0 && errno == EINTR)) continue;
    ssize_t n = read(fd, buf + used, sizeof(buf) - 1 - used);
    if (n <= 0) { /* EOF / HUP : fermeture et reconnexion */ }
    /* decoupage par '\n' puis nav_handle_line() pour chaque ligne */
}
```

Le port est ouvert avec `O_NOCTTY` pour eviter d'en faire le terminal de contrôle du processus. Le `poll()` de 500 ms (au lieu de `fgets()` bloquant) permet au thread de s'arreter rapidement a la fermeture du dashboard.

`nav_handle_line()` parse la ligne hors verrou dans une copie locale, puis la recopie dans `g_state.nav`. Le terminal affiche une ligne a chaque changement :

```
Nav updated: 300 m | Tournez a droite sur Rue X | 12 min - 4,5 km - ...
```

9.3.2 Parsing JSON

`parse_nav_json()` utilise `json_tokener_parse()` de `json-c`. Elle extrait `next_action`, `distance_to_next_`
`eta`, `street_name`, `speed_kmh`, `lat`, `lon` et remplit `nav_info_t`. La ligne du bas affiche `street_name` s'il est fourni, sinon l'ETA.

9.4 Thread musique : `music_thread_fn()`

9.4.1 Connexion D-Bus et decouverte du lecteur

`dbus_open_private("music")` ouvre une connexion privee au bus systeme (nouvel essai toutes les 2 s si le bus est indisponible).

`find_player_path()` envoie `GetManagedObjects` à `org.bluez` sur `/` et cherche le premier objet exposant `org.bluez.MediaPlayer1`. Le chemin resultant est de la forme :

```
/org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA/player0
```

Redecouverte automatique (v4) : tant qu'aucun lecteur n'est connu, la recherche est relancee toutes les 3 s (`REDISCOVER_PERIOD_S`). Si `GetAll` echoue (telephone deconnecte, `player0` devenu `player1`...), le chemin est oublie et la recherche reprend ; l'UI affiche « No MediaPlayer / Searching... ». Il n'est donc plus necessaire de relancer le dashboard apres le premier appairage.

9.4.2 Lecture des proprietes

Toutes les secondes, `get_music_properties()` appelle `GetAll("org.bluez.MediaPlayer1")` et extrait :

— `Status` (chaîne) → `g_state.music.status`
— `Position` (uint32, ms) → `g_state.music.position_ms`
— `Track` (dict) → `Title`, `Artist`, `Album`, `Duration`

En cas de changement de piste (couple titre+artiste), une demande est transmise au **thread jaquette** (`cover_thread_fn()`, file d'une requete protegee par `g_cover_lock` et `g_cover_cond`). Le telechargement, qui peut prendre plusieurs secondes, ne bloque ainsi ni les boutons musique ni l'UI. Une fois la jaquette ecrite, `g_state.cover_version` est incremente.

9.4.3 Commandes de contrôle

Les boutons LVGL ne font plus d'appel D-Bus : ils posent une commande dans `g_state.media_cmd`. Le thread musique la lit a chaque tour (toutes les 200 ms) et appelle `send_player_method()`, qui envoie `Play`, `Pause`, `Next` ou `Previous` à `org.bluez.MediaPlayer1` (timeout 3 s).

9.5 Thread appels : `calls_thread_fn()`

9.5.1 Decouverte du modem HFP

`find_modem_with_voicecall()` interroge `org.ofono.Manager.GetModems()` et selectionne le modem dont la liste d'interfaces contient `"org.ofono.VoiceCallManager"` :

`/hfp/org/bluez/hci0/dev_2C_BE_EB_7F_2D_FA`

Comme pour la musique, la recherche est relancee toutes les 3 s tant qu'aucun modem n'est trouve, et apres tout echec de `GetCalls`. Si un modem HFP est present mais eteint (`Powered = false`), il est allume via `Modem.SetProperty("Powered", true)` ; `VoiceCallManager` apparait alors au tour suivant.

9.5.2 Polling des appels

Toutes les secondes, `get_calls()` envoie `VoiceCallManager.GetCalls()` et extrait pour chaque appel `State`, `LineIdentification` et `Name`. Le thread met à jour `has_incoming_call` si un appel est en etat `"incoming"` ou `"waiting"`.

9.5.3 Decrocher et raccrocher

Les boutons posent `CALL_ANSWER` ou `CALL_HANGUP` dans `g_state.call_cmd` (avec le chemin de l'appel) et masquent le panneau immediatement. Le thread appels execute ensuite `call_voicecall_method()`, qui envoie `Answer` ou `Hangup` à `org.ofono.VoiceCall` (timeout 3 s).

9.6 Timer UI (100 ms)

Listing 16 – main.c — callback du timer LVGL (v4)

```
static void ui_update_timer(lv_timer_t *timer) {
    /* Copie rapide sous verrou, puis travail LVGL sans verrou */
    pthread_mutex_lock(&g_state.lock);
    nav = g_state.nav; music = g_state.music; call = g_state.current_call;
    cover_version = g_state.cover_version; /* ... */
    pthread_mutex_unlock(&g_state.lock);

    update_nav_display(&nav, nav_connected);
    update_music_display(&music, cover_version);
    update_call_display(incoming, &call);
    speedo_set_speed((int32_t)nav.speed);
    bt_agent_get_status(status, sizeof(status)); /* texte sous Pair */
}
```

`update_nav_display()` Instruction, distance (« -- m » si inconnue, sinon m arrondis a 10 m au-dela de 100 m, ou km), puis nom de rue ou ETA sur la ligne du bas.

`update_music_display()` Titre, artiste, label Play/Pause. La jaquette n'est rechargee que si `cover_version` a change (en v3 : decodage JPEG toutes les 100 ms), apres `lv_image_cache_drop()` pour ne pas reafficher l'ancienne image en cache.

`update_call_display()` Affiche/masque le panneau (`LV_OBJ_FLAG_HIDDEN`), nom ou « Unknown » et numero de telephone.

`speedo_set_speed()` Compteur mis à jour avec la vitesse en km/h entier.

Tous les textes passent par `label_set_if_changed()`, qui n'appelle `lv_label_set_text()` que si le texte a change (evite des redessins inutiles).

9.6.1 Accents et polices LVGL

Les polices Montserrat integrees a LVGL ne couvrent que l'ASCII : « Tournez à droite » s'afficherait « Tournez droite ». `fold_to_ascii()` decode l'UTF-8 et remplace les caracteres accentues par leur equivalent sans accent (« é » → « e », « ç » → « c », « œ » → « oe »), les espaces insecables par des espaces, « · » par « - », et ignore les emoji. Elle est appliquee aux textes de navigation et de musique via `label_set_text_ascii()`.

Pour afficher les vrais accents, il faudrait generer une police Montserrat incluant la plage Latin-1 avec le convertisseur de polices LVGL (`https://lvgl.io/tools/fontconverter`).

9.7 Layout LVGL

Widget	Taille	Position	Fond
Compteur (speedo)	1080 × 1080	Plein ecran	transparent
Conteneur nav.	500 × 500	Centre, +100 px Y	`#1a1a2e`
Conteneur musique	830 × 220	Bas, −150 px Y	`#0f0f1e`
Jaquette album	100 × 100	Gauche musique	—
Panneau appel	600 × 250	Par-dessus musique	`#393a4f`
Pair Button	60 × 60	Bas, −20 px Y	Bleu
Statut appairage	Texte	Bas, −100 px Y	—

Le panneau d'appel est cree avec `LV_OBJ_FLAG_HIDDEN` et n'est rendu visible que lors d'un appel entrant. Le Pair Button appelle `bt_agent_open_pairing_window()` ; le label de statut affiche le compte a rebours puis « Paired : <nom> ».

9.8 Callbacks des boutons

Six boutons LVGL sont definis. Leurs callbacks sont non bloquants : ils posent une commande que le thread concerne execute.

Bouton	Action (thread UI)	Execute par
Pair	`bt_agent_open_pairing_window()`	thread agent (`Adapter1`)
Precedent	`MEDIA_PREV`	thread musique (`MediaPlayer1.Previous`)
Play/Pause	`MEDIA_PLAY` / `MEDIA_PAUSE`	thread musique
Suivant	`MEDIA_NEXT`	thread musique (`MediaPlayer1.Next`)
Decrocher	`CALL_ANSWER`	thread appels (`VoiceCall.Answer`)
Raccrocher	`CALL_HANGUP`	thread appels (`VoiceCall.Hangup`)

10 Compteur de vitesse LVGL : `speedo_interactive.c/.h`

10.1 API publique

Listing 17 – speedo_interactive.h

```
void speedo_interactive_create(void);
void speedo_set_speed(int32_t speed);
```

`speedo_interactive_create()` doit être appelee une seule fois dans `create_ui()`. `speedo_set_speed()` est appelee depuis le timer UI à chaque tick de 100 ms.

10.2 Parametres du cadran

Listing 18 – speedo_interactive.c — constantes

```
#define SPEEDO_W 1080 /* Largeur totale (px) */
#define SPEEDO_H 1080 /* Hauteur totale (px) */
#define SPEED_MIN 0 /* Vitesse minimale (km/h) */
#define SPEED_MAX 260 /* Vitesse maximale (km/h) */
#define ARC_SWEEP 180 /* Balayage angulaire total (degres) */
#define ARC_START 270 /* Angle de depart (bas du cercle) */
```

10.3 Conversion vitesse/angle et aiguille

`speed_to_angle()` calcule l'angle par interpolation lineaire :

$$\theta = \texttt{ARC_START} + \frac{v - v_{\min}}{v_{\max} - v_{\min}} \times \texttt{ARC_SWEEP}$$

`update_needle()` convertit θ en radians, calcule les deux extremites de l'aiguille sur la couronne externe du cadran et met à jour l'objet ligne LVGL `needle_line`.

`create_ticks_and_labels()` cree pour chaque pas de 5 km/h :

— Graduation majeure tous les 10 km/h (plus longue, couleur claire).

— Graduation mineure tous les 5 km/h.

— Label numerique (police `lv_font_montserrat_18`) tous les 25 km/h.

`speedo_interactive_create()` cree les graduations, l'aiguille, et deux labels centraux : la valeur numerique (police 48 pt) et l'unite « km/h ».

11 Recuperation de jaquette : `album_art.c`

11.1 Signature

```
int download_itunes_cover(const char *artist,
                          const char *album,
                          const char *out_path);
```

Dependances : `libcurl` et `json-c`. Retourne 1 en succes, 0 sinon.

11.2 Flux d'execution

1. Encodage URL du terme (*artiste* + espace + *album*) via `curl_easy_escape()`.
2. Requête GET vers : `https://itunes.apple.com/search?term=<query>&entity=album&limit=1`
3. Accumulation de la reponse JSON en memoire via `write_to_memory()` (structure `Memory` avec `realloc()` dynamique).
4. Extraction de `"artworkUrl100"` dans `results[0]`.
5. Telechargement de l'image vers `out_path` via `write_to_file()`.

Si le telechargement echoue (pas de reseau, artiste inconnu), la fonction retourne 0 et l'ancienne jaquette reste affichee jusqu'au prochain changement de piste.

Modifications v4 :

— `curl_global_init()` est appele une seule fois dans `main()` (cette fonction n'est pas thread-safe) et n'est plus appele a chaque telechargement.

— `CURLOPT_NOSIGNAL` (obligatoire en multi-thread), `CURLOPT_CONNECTTIMEOUT` de 5 s et `CURLOPT_TIMEOUT` de 10 s (JSON) / 15 s (image) : sans reseau, le thread jaquette ne reste plus bloque et n'empeche pas l'arret.
— L'image est ecrite dans `cover.jpg.part` puis renommee (`rename()`) : l'UI ne lit jamais une image a moitie telechargee.
— La recherche utilise l'album s'il est connu, sinon le titre.

12 Ordre de demarrage complet

1. **Demarrer le Raspberry Pi** : `bluetooth`, `ofono` et `nav-bind` demarrent automatiquement via systemd (apres SETUP.sh).
2. **Sur le telephone** : activer le Bluetooth.
3. **Lancer le dashboard** : `./main` depuis `lv_port_pc_vscode/bin`.
4. **Premiere fois seulement** : cliquer sur **Pair** puis appairer depuis les Parametres Bluetooth *du telephone* (aucune action sur le Pi). Ensuite, le telephone se reconnecte automatiquement.
5. **Ouvrir NavRelay** et appuyer sur **Start** ; verifier l'ecran d'etat.
6. **Lancer Google Maps** et une navigation.
7. L'ordre n'a plus d'importance : le lecteur musique, le modem d'appels et `/dev/rfcomm1` sont redecouverts automatiquement.

Thread	Source de donnees	Intervalle
`nav_thread_fn`	`/dev/rfcomm1` (JSON)	`poll()` 500 ms
`music_thread_fn`	D-Bus `MediaPlayer1`	1 s (commandes : 200 ms)
`cover_thread_fn`	API iTunes (libcurl)	a chaque changement de piste
`calls_thread_fn`	D-Bus `VoiceCallManager`	1 s (commandes : 200 ms)
Thread agent (`bt_agent.c`)	D-Bus `Agent1` / `Adapter1`	200 ms
Timer UI	`lv_timer_handler()`	100 ms

13 Verification et validation pre-lancement

13.1 Checklist systeme (apres SETUP.sh)

✓ Bluetooth demarre : `sudo systemctl status bluetooth`
✓ oFono demarre : `sudo systemctl status ofono`
✓ nav-bind service actif : `sudo systemctl status nav-bind.service`
✓ Adaptateur visible : `hciconfig` (affiche `hci0`)
✓ Widget Bluetooth retire de la barre des taches (`bash disable_panel_bluetooth.sh`)

13.2 Checklist appairage

✓ Telephone detectable : `bluetoothctl scan on`
✓ Telephone appaire : `bluetoothctl paired-devices`
✓ Telephone de confiance : `bluetoothctl info <MAC>` → `Trusted: yes` (fait automatiquement par l'agent)
✓ Telephone connecte : `bluetoothctl info <MAC>` → `Connected: yes`

13.3 Checklist application

- ✓ Executable compile : `ls -la lv_port_vscode/bin/main`
- ✓ Affichage disponible : `echo $DISPLAY` → `:0`
- ✓ Groupes : `groups pi5` → contient `dialout` et `bluetooth`
- ✓ NavRelay : les 4 lignes d'etat indiquent `OK`, `bound`, `Connected to ...` et un compteur qui augmente

14 Depannage general

14.1 « Waiting for route... » en permanence

- — Verifier que `/dev/rfcomm1` existe : `ls /dev/rfcomm*`
- — Tester la reception : `cat /dev/rfcomm1` (doit afficher des lignes JSON).
- — Verifier les permissions : `sudo usermod -aG dialout pi5`.
- — Dans NavRelay : cliquer sur **Start**, puis lire l'ecran d'etat :
  - — ligne 2 « NOT bound » : desactiver puis reactiver NavRelay dans *Acces aux notifications* (frequent apres une reinstallation) ;
  - — ligne 3 bloquee sur « Waiting for the Pi... » : verifier `journalctl -fu nav-bind` (doit afficher `bound /dev/rfcomm1 ...`) ;
  - — ligne 4 qui augmente mais rien sur le dashboard : tester `cat /dev/rfcomm1` dashboard ferme.
- — Le terminal du dashboard doit afficher `Connected to /dev/rfcomm1` puis des lignes `Nav updated: ....`

14.2 Distance « -- m » ou action incorrecte

- — Afficher ce que Maps envoie : `adb logcat -s NavRelay` pendant une navigation.
- — Les lignes `title=... text=... sub=...` montrent la disposition de la notification ; la ligne `-> next=... dist=... eta=...` montre le classement.
- — Si la disposition de Maps a change, adapter `DIST_RE` / `ETA_RE` dans `MapsNotificationListener.kt`.

14.3 « No MediaPlayer »

- — Verifier A2DP avec `busctl tree org.bluez` (chercher `/player0`).
- — Jouer un morceau sur le telephone pour que BlueZ cree `MediaPlayer1`.
- — Le dashboard le detecte seul en moins de 3 s (message `[Music] MediaPlayer found`) ; il n'est plus necessaire de le redemarrer.

14.4 Aucun appel detecte

- — Verifier oFono : `systemctl status ofono`.
- — Verifier les messages `[Calls]` dans le terminal du dashboard (`HFP modem found`, `Powered on modem`).
- — Verifier l'existence de `/etc/pipewire/pipewire-pulse.conf.d/10-ofono.conf`.
- — Tester : `busctl call org.ofono / org.ofono.Manager GetModems`.

14.5 La jaquette ne s'affiche pas

- — Tester la connectivite : `curl "https://itunes.apple.com/search?term=test&entity=album&limit=1`
- — Verifier que `/home/pi5/dashboard/` est accessible en ecriture.
- — S'assurer que `libcurl` et `libjson-c` sont lies à la compilation.

14.6 Pair Button ne fonctionne pas

— Lire les messages `[BT agent]` du dashboard : `Registered (NoInputNoOutput)` doit apparaitre au demarrage.
— `RegisterAgent failed` ou `Set ... failed` : verifier que `pi5` est dans le groupe `bluetooth` (`groups pi5`), puis se reconnecter.
— `... -> rejected (press Pair first)` : la demande est arrivee hors de la fenetre de 3 minutes ; rappuyer sur Pair.
— « Bluetooth error » sous le bouton : l'adaptateur n'a pas pu etre rendu visible (`rfkill list`, `bluetoothctl show`).

14.7 Une fenetre Bluetooth du bureau apparait encore

— Fenetre « Pairing successful / Connection successful » : c'est le widget de la barre des taches ; lancer `bash disable_panel_bluetooth.sh` (sans sudo).
— Fenetre de confirmation de code : un autre agent a repris la place d'agent par defaut (par exemple `bluetoothctl default-agent`) ; le dashboard la reprend en 15 s maximum.

14.8 Le dashboard ne se ferme pas

— Utiliser Échap, Ctrl+Q, Ctrl+C ou `systemctl --user stop dashboard`.
— Un message `Thread ... did not stop in time` indique un thread bloque ; le processus est de toute facon termine apres 6 s (`alarm`).

14.9 Commandes de logs utiles

```
journalctl -fu nav-bind.service # Daemon RFCOMM
journalctl -fu dashboard.service # Application LVGL
journalctl -fu bluetooth.service # BlueZ
journalctl -fu ofono.service # oFono
dbus-monitor --system \
  "type=signal,sender=org.bluez" # Evenements BlueZ
dbus-monitor --system \
  "type=signal,sender=org.ofono" # Evenements oFono
```

15 Limitations connues

Nom de l'appelant (HFP) La propriete `Name` de `org.ofono.VoiceCall` est generalement vide avec les telephones Android modernes. Seul le numero (`LineIdentification`) est affiche de maniere fiable.

Jaquette Bluetooth (AVRCP Cover Art) BlueZ ne supporte pas AVRCP 1.6 de maniere stable. La jaquette est recuperee via l'API iTunes Search, ce qui necessite une connexion Internet et introduit un delai au changement de piste.

Format des notifications Google Maps NavRelay depend de la disposition des notifications de Maps, qui n'est pas documentee et peut changer. Le classement par contenu (ETA / distance / action) et le filet de securite limitent l'impact ; `adb logcat -s NavRelay` permet de diagnostiquer.

Accents Les polices LVGL integrees etant ASCII, les accents sont remplaces par la lettre simple.

Canal RFCOMM variable Le canal assigne à `NavJsonService` par Android peut changer à chaque redemarrage. Ce cas est gere automatiquement par `nav-bind-daemon.sh` via le registre SDP.

Widget Bluetooth du bureau Il est retire de la barre des taches pour eviter ses fenetres ; l'appairage se fait uniquement par le bouton Pair (ou `bluetoothctl`).

`sdptool nav-bind-daemon.sh` utilise `sdptool`, deprecie dans BlueZ 5 ; il necessite que `bluetoothd` tourne en mode compatibilite sur certaines installations.

16 Fichiers du projet

16.1 Fichiers essentiels

`SETUP.sh` Script d'installation une fois (4 KB). À executer apres la premiere installation, installe tous les paquets et services systemd.

`QUICK_START.md` Guide d'utilisation pour l'utilisateur final. À consulter pour le demarrage quotidien et les procedures courantes.

`nav-bind-daemon.sh` Daemon de liaison RFCOMM. Maintient le binding entre `/dev/rfcomm1` et le service NavRelay du telephone. À adapter pour un autre MAC.

`lv_port_pc_vscode/src/bt_agent.c` / `bt_agent.h` Agent d'appairage BlueZ integre au dashboard (bouton Pair). Remplace `bt_pairing.h`.

`disable_panel_bluetooth.sh` Retire (ou restaure avec `--restore`) le widget Bluetooth de la barre des taches Pi OS. Appele par `SETUP.sh`.

`bt_make_discoverable.sh` *Plus utilise par le dashboard depuis la v4* (remplace par `bt_agent.c`). Conserve pour un usage manuel.

`bt_setup_audio.sh` Configuration optionnelle de PulseAudio pour Bluetooth. Generalement non necessaire apres SETUP.sh.

`cover.jpg` Cache pour la jaquette de l'album courant (vide initialement).